

**WRITE UP**

**Experiment Number: 03**

**TITLE: Searching and Sorting**

**PROBLEM STATEMENT:**

Write a C program to create a student database using an array of structures. Apply searching (Linear and Binary Search) and sorting techniques (Insertion Sort, Selection Sort, Shell sort).

**OBJECTIVE:**

1. To understand use of an array of structures for maintaining records
2. To implement and analyze searching techniques
3. To implement and analyze sorting techniques
4. To Compare Insertion Sort, Selection Sort, Shell sort.

**THEORY:**

Searching and sorting are two of the most fundamental operations in data structures and algorithms. They are widely used in computer applications such as database management systems, search engines, operating systems, e-commerce platforms, and information retrieval systems. Efficient searching and sorting techniques improve the performance of software by reducing the time required to locate and organize data.

**Searching**

Searching is the process of finding the location of a specific element, called the **key**, within a collection of data. If the element is present, the search operation returns its position; otherwise, it indicates that the element is not found.

Searching is an essential operation in applications such as student record management, library systems, banking systems, inventory management, and online shopping.

**Types of Searching**

## **Linear Search**

Linear search, also known as Sequential Search, is the simplest searching technique. In this method, each element of the list is compared one by one with the target element until a match is found or the entire list has been searched.

## **Binary Search**

Binary Search is an efficient searching algorithm that works only on sorted data. It repeatedly divides the search interval into two halves until the target element is found or the search space becomes empty.

## **Sorting**

Sorting is the process of arranging data elements in a specific order, usually ascending or descending. Sorted data allows faster searching, easier analysis, and efficient data processing.

## **Types of Sorting**

### **Bubble Sort**

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position, similar to bubbles rising to the surface.

### **Selection Sort**

Selection Sort repeatedly selects the smallest element from the unsorted portion of the list and places it at the beginning.

### **Insertion Sort**

Insertion Sort builds a sorted list one element at a time by inserting each new element into its correct position in the already sorted portion of the array.

### **Shell Sort**

Shell Sort allows the exchange of elements that are far apart. It sorts elements by comparing and swapping items separated by a specific gap, gradually reducing the gap until it becomes 1.

### **Merge Sort**

Merge Sort is a divide-and-conquer algorithm that divides the array into smaller subarrays, sorts each subarray recursively, and then merges them into a sorted array.

## Quick Sort

Quick Sort is another divide-and-conquer algorithm. It selects a pivot element and partitions the array into two parts such that elements smaller than the pivot are placed before it and larger elements after it. The process is repeated recursively

### IMPLEMENTATION:

- **PLATFORM:**

- 64-bit Open source Linux or its derivatives.
- Open Source C Programming tool like gcc/Eclipse Editor.

- **TEST CONDITIONS: -**

1. Input at least five records.
2. Search a student in the record as per the roll number (linear search)
3. Arrange list of students roll call listwise. (Use Insertion sort, Selection sort, Shell sort)
4. Search a student as per roll number in the sorted list (Binary search)
5. Test the searching and sorting methods for at least five records.

- **PSEUDO CODE:**      *//To be Written by Students*

Write pseudocode Linear search, Binary search, Insertion sort, Selection and Shell sort.

- **TIME COMPLEXITY:**      *//To be Written by Students*

Find out time complexity of above operations

- **CONCLUSION:**

Thus, implemented different searching and sorting methods on the student database. This System is able to perform sorting under different cases.

- **FAQs**      *//To be Written by Students*

1. What is the meaning of database? How to maintain in C
2. What are the applications of searching?
3. Compare linear search and binary search?

4. What are the applications of sorting?
5. Compare and contrast, selection, insertion sort and shell sort?
6. What is external & internal sorting, In-place, Stable Features of Sorting Algorithms?

### ● PRACTICE ASSIGNMENTS

1. Write a program to insert a new record into existing record
2. Write a program to delete a record from existing record
3. Write a program to arrange list of students in alphabetical order (Use Selection Sort )

## Output:

```

riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter the number of students: 2

Enter details for student 1:
Roll Number : 45
Name : Rohit
Percentage : 98
Division : A

Enter details for student 2:
Roll Number : 18
Name : Virat
Percentage : 89
Division : B

Enter 1- selection sort, 2-insertion sort, 3-shell sort1
Enter 1-linear search, 2-binary search2

Data sorted using Selection Sort!

Roll No : 18
Name : Virat
Percentage : 89.00
Division : B
Roll No : 45
Name : Rohit
Percentage : 98.00
Division : A
Enter roll number for searching:45

[Binary Search] Target found!
Name : Rohit
Percentage : 98.00
Division : A
riamudhole@Ubuntu:~/Desktop/DS Sem 3$

riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter the number of students: 3

Enter details for student 1:
Roll Number : 45
Name : Rohit
Percentage : 98
Division : A

Enter details for student 2:
Roll Number : 18
Name : Virat
Percentage : 89
Division : B

Enter details for student 3:
Roll Number : 77
Name : Shubman
Percentage : 65
Division : C

Enter 1- selection sort, 2-insertion sort, 3-shell sort2
Enter 1-linear search, 2-binary search1

Data sorted using Insertion Sort!

Roll No : 18
Name : Virat
Percentage : 89.00
Division : B
Roll No : 45
Name : Rohit
Percentage : 98.00
Division : A
Roll No : 77
Name : Shubman
Percentage : 65.00
Division : C
Enter roll number to search: 79

Roll number 79 not found.
riamudhole@Ubuntu:~/Desktop/DS Sem 3$

```

```

Enter details for student 2:
Roll Number : Name      : Percentage : ^Z
[2]+  Stopped              ./a.out
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter the number of students: 3

Enter details for student 1:
Roll Number : 45
Name       : Rohit
Percentage : 98
Division  : M

Enter details for student 2:
Roll Number : 18
Name       : Virat
Percentage : 83
Division  : R

Enter details for student 3:
Roll Number : 17
Name       : Rishabh
Percentage : 93
Division  : D
Enter 1- selection sort, 2-insertion sort, 3-shell sort3
Enter 1-linear search, 2-binary search3

Data sorted using Shell Sort!

Roll No   : 17
Name      : Rishabh
Percentage : 93.00
Division  : D
Roll No   : 18
Name      : Virat
Percentage : 83.00
Division  : R
Roll No   : 45
Name      : Rohit
Percentage : 98.00
Division  : MEnter valid choice for searchingriamudhole@Ub

```

## Code:

```
#include <stdio.h>
```

```
struct stud
```

```
{
    int roll;
    char name[20];
    float percent;
    char div;
};
```

```
void accept(struct stud s[], int n)
```

```
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("\nEnter details for student %d:\n", i + 1);
        printf("Roll Number : ");
        scanf("%d", &s[i].roll);
        printf("Name       : ");
        scanf("%s", s[i].name);
        printf("Percentage : ");
        scanf("%f", &s[i].percent);
        printf("Division  : ");
        scanf(" %c", &s[i].div);
    }
}
```

```

}
void display(struct stud s[], int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("\nRoll No   : %d", s[i].roll);
        printf("\nName      : %s", s[i].name);
        printf("\nPercentage : %.2f", s[i].percent);
        printf("\nDivision   : %c", s[i].div);
    }
}
void linear_search(struct stud s[], int n, int t)
{
    int i, found = 0;
    for (i = 0; i < n; i++)
    {
        if (s[i].roll == t)
        {
            printf("\n[Linear Search] Target found!");
            printf("\nName      : %s", s[i].name);
            printf("\nPercentage : %.2f", s[i].percent);
            printf("\nDivision   : %c\n", s[i].div);
            found = 1;
            break;
        }
    }
    if (!found)
    {
        printf("\nRoll number %d not found.\n", t);
    }
}
void binary_search(struct stud s[], int n, int t)
{
    int low = 0, high = n - 1, mid;
    int found = 0;

    while (low <= high)
    {
        mid = low + (high - low) / 2;

        if (s[mid].roll == t)
        {
            printf("\n[Binary Search] Target found!");
            printf("\nName      : %s", s[mid].name);
            printf("\nPercentage : %.2f", s[mid].percent);
            printf("\nDivision   : %c\n", s[mid].div);
            found = 1;
            break;
        }
        else if (s[mid].roll < t)
        {
            low = mid + 1;
        }
    }
}

```

```

        else
        {
            high = mid - 1;
        }
    }

    if (!found)
    {
        printf("\nRoll number %d not found.\n", t);
    }
}

void selection_sort(struct stud s[], int n)
{
    int i, j, min_idx;
    struct stud temp;

    for (i = 0; i < n - 1; i++)
    {
        min_idx = i;
        for (j = i + 1; j < n; j++)
        {
            if (s[j].roll < s[min_idx].roll)
            {
                min_idx = j;
            }
        }
        if (min_idx != i)
        {
            temp = s[i];
            s[i] = s[min_idx];
            s[min_idx] = temp;
        }
    }
    printf("\nData sorted using Selection Sort!\n");
}

void insertion_sort(struct stud s[], int n)
{
    int i, j;
    struct stud key;

    for (i = 1; i < n; i++)
    {
        key = s[i];
        j = i - 1;

        while (j >= 0 && s[j].roll > key.roll)
        {
            s[j + 1] = s[j];
            j = j - 1;
        }
        s[j + 1] = key;
    }
    printf("\nData sorted using Insertion Sort!\n");
}

```

```

void shell_sort(struct stud s[], int n)
{
    int gap, i, j;
    struct stud temp;

    for (gap = n / 2; gap > 0; gap /= 2)
    {
        for (i = gap; i < n; i++)
        {
            temp = s[i];
            for (j = i; j >= gap && s[j - gap].roll > temp.roll; j -= gap)
            {
                s[j] = s[j - gap];
            }
            s[j] = temp;
        }
    }
    printf("\nData sorted using Shell Sort!\n");
}

int main()
{
    int n, main_ch, sub_ch, t;

    printf("Enter the number of students: ");
    scanf("%d", &n);
    struct stud s[n];
    accept(s,n);
    printf("Enter 1- selection sort, 2-insertion sort, 3-shell sort");
    scanf("%d", &main_ch);
    printf("Enter 1-linear search, 2-binary search");
    scanf("%d", &sub_ch);
    switch(main_ch)
    {
        case 1:
            selection_sort(s,n);
            display(s,n);
            break;
        case 2:
            insertion_sort(s, n);
            display(s, n);
            break;
        case 3:
            shell_sort(s,n);
            display(s,n);
            break;
        default:
            printf("Enter valid choice for sorting");
            break;
    }
    switch(sub_ch)
    {
        case 1:
            printf("\nEnter roll number to search: ");
            scanf("%d", &t);

```

```
    linear_search(s, n, t);  
    break;  
    case 2:  
        printf("\nEnter roll number for searching:");  
        scanf("%d", &t);  
        binary_search(s,n,t);  
        break;  
    default:  
        printf("Enter valid choice for searching");  
    }  
    return 0;  
}
```